Giải thích chi tiết Code AMR - Phần 2: Odometry, PID, WiFi, Bluetooth

> Tiếp nối Phần 1. Đây là phần lõi thuật toán điều khiển.

PHASE 5: ODOMETRY — Tính toán vị trí (Odometry.ino)

5.1 Hằng số tính toán

```
const float WHEEL_CIRCUMFERENCE = PI * WHEEL_DIAMETER;
// = 3.14159 × 66.0 = 207.35mm (chu vi bánh xe)

const float DISTANCE_PER_TICK = WHEEL_CIRCUMFERENCE / ENCODER_PPR;
// = 207.35 / 495 = 0.4188mm (quãng đường mỗi tick encoder)
```

5.2 `Odom_init()` — Reset về gốc tọa độ

```
void Odom_init() {
  odom_x = 0.0;       // Vị trí X = 0mm
  odom_y = 0.0;       // Vị trí Y = 0mm
  odom_theta = 0.0;   // Góc hướng = 0 rad
  prev_encoder1 = encoder1_value;  // Lưu giá trị encoder hiện tại
  prev_encoder2 = encoder2_value;  // làm mốc ban đầu
  odom_prevTime = micros();        // Lưu thời điểm hiện tại
}
```

5.3 `Odom_update()` — Cập nhật tọa độ (gọi mỗi 20ms)

Tính thời gian trôi qua:

```
void Odom_update() {
  unsigned long now = micros();
  float dt = (now - odom_prevTime) / 1000000.0;  // Microseconds → Seconds
  odom_prevTime = now;
  if (dt <= 0 || dt > 1.0) dt = 0.02;  // Bảo vệ: nếu bất thường → dùng 20ms
```

Tính delta ticks (số xung thay đổi):

```
  int delta_tickL = (encoder1_value - prev_encoder1);  // Xung mới - xung cũ trái
  int delta_tickR = (encoder2_value - prev_encoder2);  // Xung mới - xung cũ phải
```

```
  prev_encoder1 = encoder1_value;  // Cập nhật mốc cho lần sau
  prev_encoder2 = encoder2_value;
```

Tính vận tốc từng bánh:

```
  // v = (delta_tick × π × D) / (N × dt)
  vLeft  = (delta_tickL * PI * WHEEL_DIAMETER) / (ENCODER_PPR * dt);
  vRight = (delta_tickR * PI * WHEEL_DIAMETER) / (ENCODER_PPR * dt);
  // Ví dụ: 10 ticks trong 0.02s → vLeft = (10 × 3.14 × 66) / (495 × 0.02) = 209 mm/s

  v = (vLeft + vRight) / 2.0;      // Vận tốc tịnh tiến (mm/s)
  w = (vLeft - vRight) / WHEEL_BASE; // Vận tốc góc (rad/s)
```

Tính quãng đường & cập nhật tọa độ:

```
  float SL = delta_tickL * DISTANCE_PER_TICK;  // Quãng đường bánh trái (mm)
  float SR = delta_tickR * DISTANCE_PER_TICK;  // Quãng đường bánh phải (mm)
  float S = (SL + SR) / 2.0;                   // Quãng đường trung bình
  float delta_theta = (SL - SR) / WHEEL_BASE;  // Thay đổi góc (rad)

  // Mid-point approximation: dùng góc giữa để tính X, Y chính xác hơn
  odom_x += S * cos(odom_theta + delta_theta / 2.0);
  odom_y += S * sin(odom_theta + delta_theta / 2.0);
  odom_theta += delta_theta;

  x_cood = -odom_x;  // Đảo dấu cho phù hợp hệ tọa độ giao diện
  y_cood = -odom_y;

  odom_theta = atan2(sin(odom_theta), cos(odom_theta));  // Chuẩn hóa [-π, π]
}
```

5.4 `velocityToPWM()` — Chuyển mm/s → PWM

```
const float MAX_VELOCITY = 594.0;  // Đo thực tế: 594 mm/s tại PWM 255
const int MIN_PWM = 50;            // PWM tối thiểu để motor quay được

int velocityToPWM(float velocity_mms) {
  if (fabs(velocity_mms) < 5.0) return 0;  // Gần 0 → tắt motor

  int pwm = (int)(velocity_mms * 255.0 / MAX_VELOCITY);
  // Ví dụ: 200 mm/s → pwm = 200 × 255 / 594 = 85

  // Đảm bảo PWM >= MIN_PWM khi có lệnh chạy (tránh dead zone motor)
  if (pwm > 0 && pwm < MIN_PWM) pwm = MIN_PWM;
  if (pwm < 0 && pwm > -MIN_PWM) pwm = -MIN_PWM;
```

```
  return constrain(pwm, -255, 255);
}
```

5.5 `driveVW(v, w)` — Điều khiển bằng (v, ω)

```
void driveVW(float v_cmd, float w_cmd) {
  // Công thức ngược Differential Drive:
  float vL = v_cmd - (w_cmd * WHEEL_BASE) / 2.0;  // Vận tốc bánh trái
  float vR = v_cmd + (w_cmd * WHEEL_BASE) / 2.0;  // Vận tốc bánh phải
  // Ví dụ xoay tại chỗ: v=0, w=2 → vL = -205, vR = +205 (2 bánh quay ngược)

  int pwmL = velocityToPWM(vL);
  int pwmR = velocityToPWM(vR);
  Motor(pwmL, pwmR);  // Gửi PWM xuống driver
}
```

5.6 `goToXY_init()` — Khởi tạo nhiệm vụ di chuyển

```
void goToXY_init(float x_target, float y_target) {
  totalDistTarget = sqrt(x_target*x_target + y_target*y_target);
  // Ví dụ: (300, 400) → sqrt(90000+160000) = 500mm

  totalDistTraveled = 0.0;
  targetAngle = atan2(y_target, x_target);  // Góc cần hướng tới
  // Ví dụ: atan2(400, 300) = 0.927 rad ≈ 53.1°

  rotationDone = false;
  prevTickL_nav = encoder1_value;
  prevTickR_nav = encoder2_value;
}
```

5.7 `goToXY()` — Hàm tự hành chính (gọi liên tục trong loop)

BƯỚC 1 — Xoay tại chỗ:

```
bool goToXY(float x_target, float y_target) {
  float dt = 0.02;

  if (!rotationDone) {
    float yaw_rad = yaw * PI / 180.0;           // Yaw IMU: độ → rad
    float angle_error = targetAngle - yaw_rad;   // Sai số góc
    angle_error = atan2(sin(angle_error), cos(angle_error)); // Chuẩn hóa

    if (fabs(angle_error) > ANGLE_THRESHOLD) {   // Chưa đúng hướng (>0.02 rad)
      float w_turn = PID_theta(targetAngle, dt); // PID tính vận tốc xoay
      driveVW(0, w_turn);                        // v=0 (đứng yên), chỉ xoay
```

```
      // Reset encoder để không tính xoay vào quãng đường
      prevTickL_nav = encoder1_value;
      prevTickR_nav = encoder2_value;
      return false;  // Chưa xong → loop gọi lại
    }

    // Đã xoay xong → chuyển sang đi thẳng
    rotationDone = true;
    odom_x = 0.0; odom_y = 0.0;        // Reset tọa độ
    IMU_reset();                         // Reset yaw về 0
    PID_resetStraight();                 // Reset tất cả PID
    PID_resetHeading();
    PID_resetTheta();
  }
```

BƯỚC 2 — Đi thẳng:

```
  // Đếm quãng đường bằng encoder
  int dTickL = -(encoder1_value - prevTickL_nav);  // Dấu âm: bù chiều quay
  int dTickR = -(encoder2_value - prevTickR_nav);
  prevTickL_nav = encoder1_value;
  prevTickR_nav = encoder2_value;

  float dSL = dTickL * DISTANCE_PER_TICK;
  float dSR = dTickR * DISTANCE_PER_TICK;
  float dS  = (dSL + dSR) / 2.0;
  totalDistTraveled += dS;

  // Đã đi đủ → DỪNG
  if (totalDistTraveled >= totalDistTarget - DIST_THRESHOLD) {
    Motor(0, 0);
    return true;  // BÁO ĐÃ ĐẾN NƠI
  }

  // Tính PWM cơ bản + giảm tốc khi gần đích
  float remaining = totalDistTarget - totalDistTraveled;
  float v_cmd = DRIVE_SPEED;           // 200 mm/s
  if (remaining < 80.0)
    v_cmd = DRIVE_SPEED * (remaining / 80.0);  // Giảm tuyến tính trong 80mm cuối
  int basePWM = velocityToPWM(v_cmd);

  // ★ KẾT HỢP 2 PID:
  float corrEncoder = PID_straight(dt);     // PID encoder: giữ 2 bánh bằng nhau
  float corrYaw = -PID_heading(dt);         // PID IMU: giữ hướng thẳng
  float totalCorr = corrEncoder + corrYaw;  // Tổng bù

  // Áp dụng: bánh lệch → bù PWM ngược chiều
  int pwmL = basePWM + (int)totalCorr;
```

```
  int pwmR = basePWM - (int)totalCorr;
  pwmL = constrain(pwmL, -255, 255);
  pwmR = constrain(pwmR, -255, 255);
  Motor(pwmL, pwmR);
  return false;  // Chưa đến → loop gọi lại
}
```

PHASE 6: BỘ ĐIỀU KHIỂN PID (PID.ino)

6.1 Thông số PID

```
// PID Xoay góc (Bước 1)
float pidTheta_Kp = 10.0;  float pidTheta_Ki = 0.0;  float pidTheta_Kd = 0.0;

// PID Giữ hướng khi đi thẳng (Bước 2 - dùng IMU)
float pidHeading_Kp = 5.0; float pidHeading_Ki = 0.0; float pidHeading_Kd = 0.0;

// PID Cân bằng encoder (Bước 2 - dùng encoder)
float pidStraight_Kp = 5.0; float pidStraight_Ki = 0.0; float pidStraight_Kd = 0.0;

const float INTEGRAL_LIMIT = 100.0;  // Chống windup
```

6.2 `PID_theta()` — Xoay góc (Bước 1)

```
float PID_theta(float setpoint_rad, float dt) {
  float actual_rad = (float)yaw * PI / 180.0;      // IMU yaw → rad
  float error = setpoint_rad - actual_rad;           // Sai số góc
  error = atan2(sin(error), cos(error));             // Chuẩn hóa [-π, π]

  pidTheta_integral += error * dt;                   // Tích phân sai số
  pidTheta_integral = constrain(pidTheta_integral, -INTEGRAL_LIMIT, INTEGRAL_LIMIT);

  float derivative = 0.0;
  if (dt > 0) derivative = (error - pidTheta_prevError) / dt;  // Đạo hàm
  pidTheta_prevError = error;

  float output = pidTheta_Kp * error        // P: tỉ lệ với sai số
               + pidTheta_Ki * pidTheta_integral  // I: tích lũy sai số
               + pidTheta_Kd * derivative;        // D: tốc độ thay đổi

  output = constrain(output, -3.0, 3.0);  // Giới hạn ω ≤ 3 rad/s
  return output;  // → truyền vào driveVW(0, output)
}
```

6.3 `PID_heading()` — Giữ hướng 0° khi đi thẳng (Bước 2)

```
float PID_heading(float dt) {
  float error = 0.0 - yaw;  // Setpoint=0, actual=yaw → lệch bao nhiêu độ?
  // Ví dụ: yaw = 3° → error = -3 → robot đang lệch phải 3°

  pidHeading_integral += error * dt;
  pidHeading_integral = constrain(pidHeading_integral, -INTEGRAL_LIMIT, INTEGRAL_LIMIT);

  float derivative = 0.0;
  if (dt > 0) derivative = (error - pidHeading_prevError) / dt;
  pidHeading_prevError = error;

  float output = pidHeading_Kp * error
               + pidHeading_Ki * pidHeading_integral
               + pidHeading_Kd * derivative;

  output = constrain(output, -100.0, 100.0);  // Giới hạn correction PWM
  return output;  // → dùng làm corrYaw trong goToXY()
}
```

6.4 `PID_straight()` — Cân bằng 2 bánh (Bước 2)

```
float PID_straight(float dt) {
  int encL = encoder1_value - straightStartEnc1;  // Tổng tick trái từ lúc bắt đầu
  int encR = encoder2_value - straightStartEnc2;  // Tổng tick phải từ lúc bắt đầu
  float error = (float)(encL - encR);             // Chênh lệch 2 bánh
  // Ví dụ: encL=500, encR=490 → error=10 → bánh trái đi nhiều hơn → lệch phải

  pidStraight_integral += error * dt;
  pidStraight_integral = constrain(pidStraight_integral, -INTEGRAL_LIMIT, INTEGRAL_LIMIT);

  float derivative = 0.0;
  if (dt > 0) derivative = (error - pidStraight_prevError) / dt;
  pidStraight_prevError = error;

  float output = pidStraight_Kp * error
               + pidStraight_Ki * pidStraight_integral
               + pidStraight_Kd * derivative;

  output = constrain(output, -100.0, 100.0);
  return output;  // → dùng làm corrEncoder trong goToXY()
}
```

> **Kết hợp 2 PID:** `totalCorr = corrEncoder + corrYaw`, sau đó `pwmL = base + corr`, `pwmR = base - corr`. Encoder bù chênh lệch vòng quay, IMU bù góc lệch tổng thể.

PHASE 7: VÒNG LẶP CHÍNH (`loop()` trong AMR_MAIN.ino)

```
void loop() {
  IMU_read();                    // 1. Đọc góc xoay từ IMU

  if (odomFlag) {               // 2. Mỗi 20ms → cập nhật odometry
    odomFlag = false;
    Odom_update();
  }

  WiFi_readCommand();            // 3. Kiểm tra lệnh từ Python

  if (!arrived && wifi_hasTarget) {  // 4. Nếu đang có mục tiêu
    arrived = goToXY(wifi_targetX, wifi_targetY);  // Di chuyển
    if (arrived) {
      wifi_hasTarget = false;    // Đã đến → xóa mục tiêu
    }
  }

  WiFi_sendStatus();             // 5. Gửi trạng thái về Python
  Odom_print();                  // 6. In Serial debug
  IMU_print();
  BT_debugAll();                 // 7. Gửi debug Bluetooth
  delay(20);                     // 8. Chờ 20ms = ~50Hz
}
```

PHASE 8: GIAO TIẾP WIFI (Wifi.ino)

8.1 `WiFi_init()` — Phát AP + TCP Server

```
void WiFi_init() {
  WiFi.softAP(WIFI_SSID, WIFI_PASS);  // SSID: "AMR_Robot_WiFi", Pass: "12345678"
  wifiServer.begin();                  // Mở TCP Server port 8080
  // IP mặc định: 192.168.4.1
}
```

8.2 `WiFi_readCommand()` — Nhận lệnh từ Python

```
void WiFi_readCommand() {
  // Kiểm tra client kết nối
  if (!wifiClient || !wifiClient.connected()) {
    WiFiClient newClient = wifiServer.available();
    if (newClient) wifiClient = newClient;
    return;
  }
```

```
  if (wifiClient.available()) {
    String cmd = wifiClient.readStringUntil('\n');
    cmd.trim();

    if (cmd.startsWith("GO,")) {          // Lệnh di chuyển: "GO,500.0,300.0"
      // Parse tọa độ X, Y từ chuỗi
      int comma1 = cmd.indexOf(',');
      int comma2 = cmd.indexOf(',', comma1 + 1);
      wifi_targetX = cmd.substring(comma1 + 1, comma2).toFloat();
      wifi_targetY = cmd.substring(comma2 + 1).toFloat();
      wifi_hasTarget = true;

      goToXY_init(wifi_targetX, wifi_targetY);  // Khởi tạo navigation
      arrived = false;                            // Bắt đầu di chuyển
      wifiClient.println("OK");                   // Phản hồi Python
    }
    else if (cmd == "STOP") {             // Lệnh dừng
      arrived = true;
      wifi_hasTarget = false;
      wifiClient.println("STOPPED");
    }
    else if (cmd == "RESET") {            // Lệnh reset
      arrived = true;
      wifi_hasTarget = false;
      wifiClient.println("RESET_OK");
    }
  }
}
```

8.3 `WiFi_sendStatus()` — Gửi trạng thái về Python

```
void WiFi_sendStatus() {
  if (!wifiClient || !wifiClient.connected()) return;

  char buf[128];
  snprintf(buf, sizeof(buf),
    "STATUS,%.1f,%.1f,%.2f,%d\n",
    x_cood, y_cood, yaw, arrived ? 1 : 0
  );
  // Ví dụ output: "STATUS,150.3,200.1,45.20,0\n"
  wifiClient.print(buf);
}
```

PHASE 9: BLUETOOTH DEBUG (Bluetooth_debug.ino)

9.1 `BT_init()` & `BT_debugAll()`

```
void BT_init() {
  SerialBT.begin("AMR_Robot");  // Phát Bluetooth với tên "AMR_Robot"
}

void BT_debugAll() {
  char buf[128];
  snprintf(buf, sizeof(buf),
    "yaw:%.1f x:%.1f y:%.1f th:%.1f e1:%d e2:%d\n",
    yaw, x_cood, y_cood,
    odom_theta * 180.0 / PI,       // Theta: rad → độ
    encoder1_value, encoder2_value
  );
  // Ví dụ: "yaw:45.2 x:150.3 y:200.1 th:44.8 e1:1200 e2:1195\n"
  SerialBT.print(buf);
}
```

> Dùng app "Serial Bluetooth Terminal" trên điện thoại để xem dữ liệu realtime.